

Today's content

✓ Pair Sum

✓ Pair Difference

→ Count of subarrays with $\text{sum} = K$.

→ Triplet Sum

→ Maximum water accumulated.

Q1) Given arr[N] with sorted distinct elements, Count all the pairs (i, j) such that $arr[i] + arr[j] = x$ and $i \neq j$.

arr[1] = [-3 0 1 3 6 8 11 14 18 25] , k=17
 0 1 2 3 4 5 6 7 8 9
 arr=2.

$K=12$, $q_{n0}=1$.

B.f idea. $\rightarrow$ Consider all the pairs. $\left[\begin{array}{l} T.C \rightarrow O(N^2) \\ S.C \rightarrow O(1) \end{array} \right]$

Idea.2. $a + b = k \Rightarrow b = k - a.$

Apply B.S to find this value.

For every element, call Binary Search for K -element.

$$\left[\begin{array}{l} T.C \rightarrow O(n \log n) \\ S.C \rightarrow O(1) \end{array} \right]$$

idea.2. → Use hashset.

→ insert all the elements in the hashset.

- For every element, check in hashset if k-element is present or not.

$$\begin{bmatrix} \text{T.C.} \rightarrow O(N) \\ \text{S.C.} \rightarrow O(N) \end{bmatrix}$$

$$\left[\begin{array}{l} T.C \rightarrow O(N) \\ S.C \rightarrow O(N) \end{array} \right]$$

arr[] = [-3 0 1 3 6 8 11 14 18 25] , K=17

_{0 1 2 3 4 5 6 7 8 9}
 $\uparrow \uparrow$
 $i \quad j$

① $i = 0, j = 1$ ✗

② $i = N-1, j = N-2$ ✗

③ $i = \frac{N}{2}, j = \frac{N}{2} + 1$ ✗

④ $i = 0, j = N-1$

arr[i] + arr[j] K

count = 0

$-3 + 25 = 22 > K$

$-3 + 18 = 15 < K$

$0 + 18 = 18 > K$

$0 + 14 = 14 < K$

$1 + 14 = 15 < K$

$3 + 14 = 17 = K$

$6 + 11 = 17 = K$

1

2

pseudo-code-

$i = 0, j = N-1;$

count = 0;

while ($i < j$) {

if ($arr[i] + arr[j] == K$) {

count++;

i++;

j--;

}

else if ($arr[i] + arr[j] < K$) {

i++;

else {

j--;

}

}

return count;

→ change this in
case of duplicates.

$T.C \rightarrow O(N)$
 $S.C \rightarrow O(1)$

Q1) Given $arr[N]$ with sorted distinct elements, count of all the pairs (i, j) such that $arr[j] - arr[i] == k$ and $i \neq j$.

$$k = |K|$$

$arr[] \rightarrow [-3, 0, 1, 3, 6, 8, 11, 14, 18, 25, 30]$, $k=5$
 $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 & 10 \end{matrix}$
ans = 4.

$arr[] \rightarrow [5, 4, 2, 12, 1, 6]$, $k=10$
 $\begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$
ans $\rightarrow (2, 3)$

i, j
 $arr[j] - arr[i] = 10$
 $arr[3] - arr[2] = 10$

idea-1 $\rightarrow$ Consider all pairs $T.C \rightarrow O(N^2)$, $S.C \rightarrow O(1)$

idea-2 $\rightarrow$ For every element, use Binary Search. $T.C \rightarrow O(N \log N)$
 $S.C \rightarrow O(1)$

idea-3 $\rightarrow$ Hashing $T.C \rightarrow O(N)$, $S.C \rightarrow O(N)$

arr[] = [-3, 0, 1, 2, 3, 6, 8, 11, 14, 18, 25, 30], k=5

_{0 1 2 3 4 5 6 7 8 9 10 11}

$\uparrow$ $\uparrow$
 i j

$$30 - (-3) = 33 > k$$

$$i = 0, j = n-1 \quad \times$$

$$i = n/2, j = n/2 + 1 \quad \times$$

increase i
or
decrease j

$$i = 0, j = 1$$

$$\text{arr}[j] - \text{arr}[i] \leq k$$

$$0 - (-3) = 3 < k$$

discarding '0' as larger element.

$$1 - (-3) = 4 < k$$

discarding '1' as larger element

$$2 - (-3) = 5 = k$$

i++, j++, count++

$$3 - 0 = 3 < k$$

discarding '3' as larger element.

$$6 - 0 = 6 > k$$

i++, discarding '6' as smaller element.

#code:-

$i = 0$, $j = 1$, $count = 0$

while($j < N$) {

int diff = arr[j] - arr[i];

if (diff == k) {

{
count++;
i++;
j++;
}

else if (diff < k) {

{
j++

else {

{
i++;
if (i == j) { j++; }
}

}

$T.C \rightarrow O(N)$
 $S.C \rightarrow O(1)$

Q.1 Given an array of +ve integers, find count of subarrays with sum = K.

arr[] $\rightarrow$ [1 2 5 4 3] $K=9$.
0 1 2 3 4

arr[] $\rightarrow$ [3 2 5 1 8 6 2 10] , $K=15$, ans=1.
0 1 2 3 4 5 6 7

B.f. idea. $\rightarrow$ Consider all the subarrays & calculate their sum.

T.C $\rightarrow O(N^3)$, S.C $\rightarrow O(1)$

idea.2 psum[]

psum[] $\rightarrow$ [3 5 10 11 19 25 27 37]
0 1 2 3 4 5 6 7

Subarray sum from i to j $\Rightarrow$ psum[j] - psum[i-1] = K.

$\therefore$ We are looking for two elements in psum[] with difference = K.

$\Rightarrow$ Count of Pair Difference.

$\left(\begin{array}{l} \text{T.C} \rightarrow O(N) \\ \text{S.C} \rightarrow O(1) \end{array} \right)$

Q1 Given arr[N] with sorted distinct elements. Find triplet (i, j, k) such that $arr[i] + arr[j] + arr[k] = sum$. ($i \neq j \neq k$)

arr $\rightarrow$ $[-8 \quad -4 \quad -3 \quad -1 \quad 2 \quad 3 \quad 5 \quad 7 \quad 9]$, sum = 14.
0 1 2 3 4 5 6 7 8

2, 5, 7

2, 3, 9

ans = 2.

Bf $\rightarrow$ consider all triplets. $\left[\begin{array}{l} T.C \rightarrow O(N^3) \\ S.C \rightarrow O(1) \end{array} \right]$

idea 2 - $a + b + c = k$

$b + c = k - a \iff$ Pair Sum Question

code $\rightarrow$

count = 0

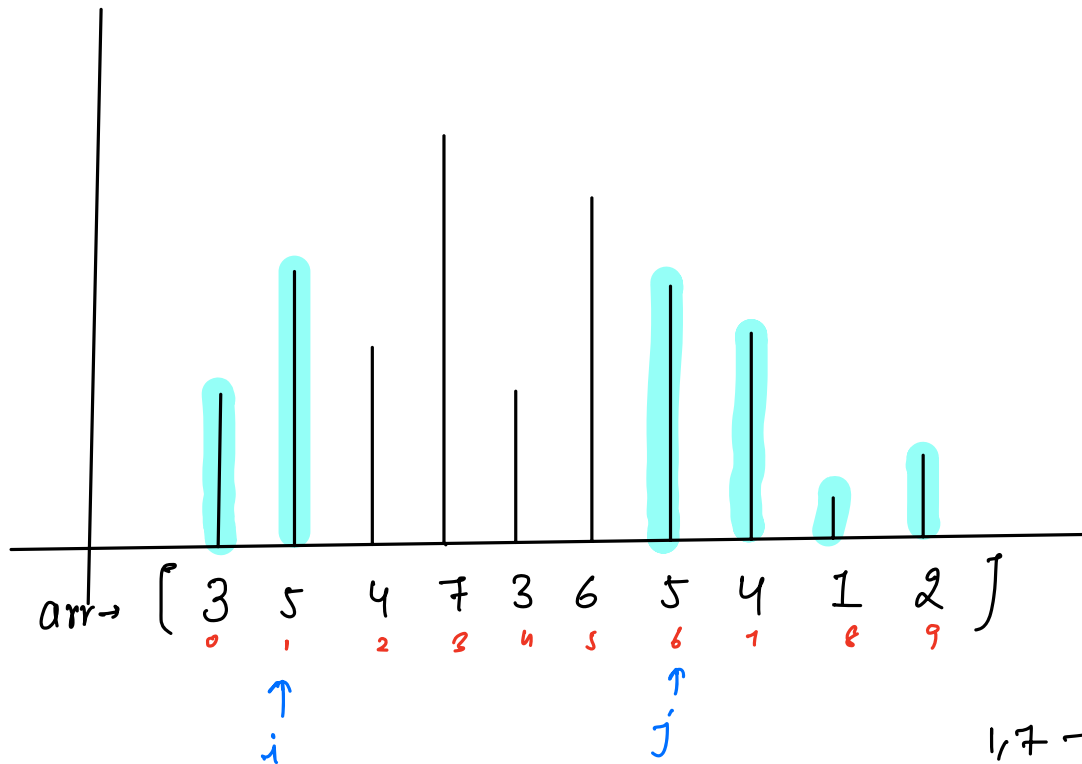
for (i = 0; i < N-2; i++) {

{
target = k - arr[i];
count += pairSum(arr, target, i+1);
}

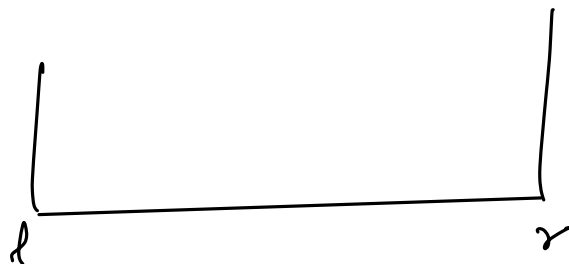
return count;

$\left[\begin{array}{l} T.C \rightarrow O(N^2) \\ S.C \rightarrow O(1) \end{array} \right]$

Q) Given $arr[N]$ where every element represents height of walls. You need to pick any two walls such that water accumulated is maximum.



$1, 7 \rightarrow 24 \times$
 $3, 5 \rightarrow 12 \times$
 $1, 6 \rightarrow 25$
 $2, 7 \rightarrow 20 \times$
 $3, 7 \rightarrow 16 \times$
 $0, 9 \rightarrow 18 \times$



Water accumulated b/w l and $r \rightarrow \min(A[l], A[r]) \times (r - l)$

B.f. $\rightarrow$ consider all the pairs $\rightarrow T.C \rightarrow O(N^2)$

idea-2 → h1alls should be as far as possible.

$$9 * 2 = 18$$

$$8 * 1 = 8$$

$$7 * 3 = 21$$

$$6 * 4 = 24$$

$$5 * 5 = 25$$

code →

ans = 0;

i = 0, j = N-1;

while (i < j) {

water = (j-i) * min (A[i], A[j]);

ans = max (ans, water);

if (A[i] < A[j]) {

{ i++;

else {

{ j--;

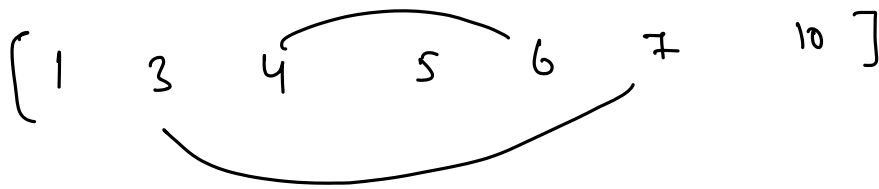
}

return ans;

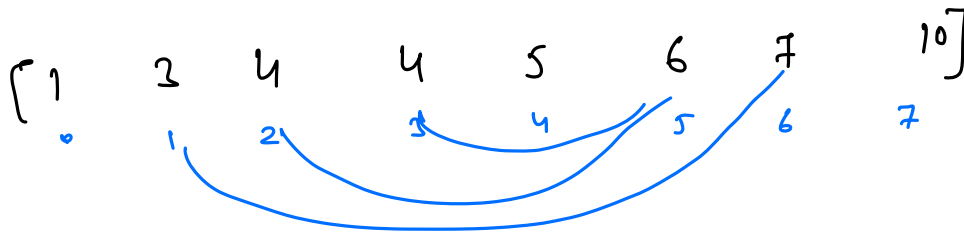
[T.C → O(N)
S.C → O(1)]

Pair-Sum. with Duplicates

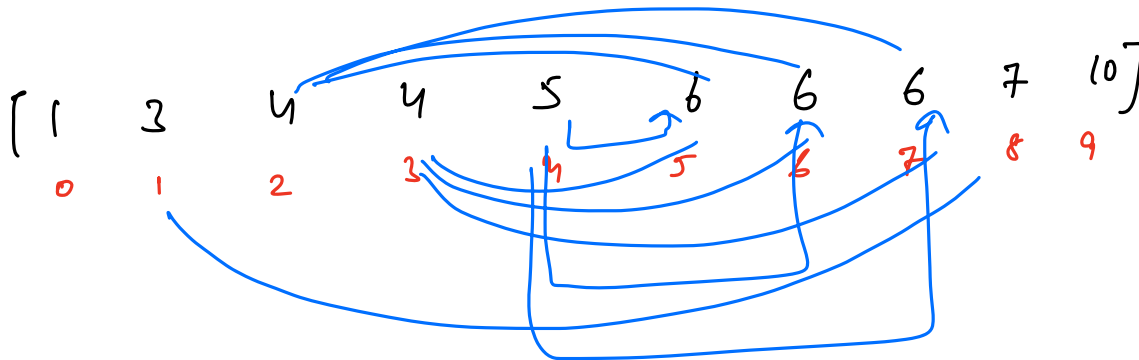
k=10



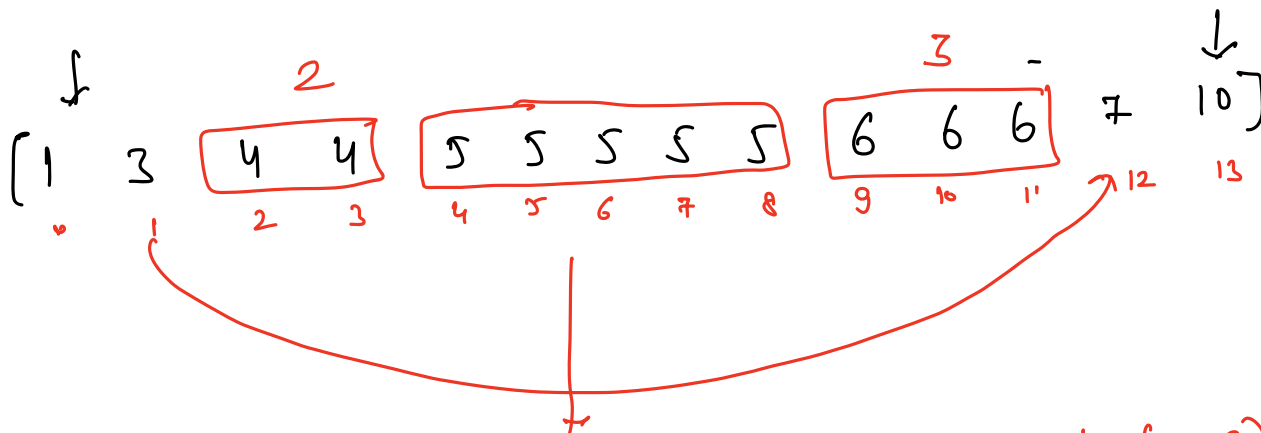
ans=2.



ans=3.



ans=7.



left = 4

LC = 12

$$SC_2 = \frac{5 \times 4}{2}$$

$$1 + (2 \times 3) + \left(\frac{5 \times 4}{2}\right)$$

$$\Rightarrow 1 + 6 + 10 = 17$$

$$N C_2 = \frac{N(N-1)}{2}$$

```
if (arr[i] + arr[j] == k) {
```

```
    if (arr[i] == arr[j]) {
```

```
        {
            count = j - i + 1;
            ans += (count * (count - 1)) / 2;
            break;
        }
```

```
    else {
```

```
        left = arr[i], lc = 0;
```

```
        while (arr[i] == left) {
```

```
            {
                lc++;
                i++;
            }
```

```
        right = arr[j], rc = 0
```

```
        while (arr[j] == right) {
```

```
            {
                rc++;
                j--;
            }
```

```
        ans += lc * rc;
```

```
    }
```

```
}
```